

Cattle IoT — Technical Document

This document captures **development-focused details** for the ESP32 cattle collar, backend, and Godhan app integration.

1. ESP32 → Cloud Communication Flow

ESP32 Collar (Sensors)

- ↓ Sample & Aggregate
- ↓ MQTT Publish (TLS, JSON payload)
- ↓ Wi-Fi AP / Farm Router
- ↓ MQTT Broker (Cloud EMQX / Mosquitto)
- ↓ Ingestion Service (Node.js / Python)
- ↓ MongoDB / InfluxDB
- ↓ Analytics Engine (Heat/Calving)
- ↓ Godhan App (APIs & Dashboards)

- **Publish Interval:** every 5 minutes (configurable).
- **Protocol:** MQTT over TLS (port 8883).
- **QoS:** 1 (at least once delivery).
- **Payload format (JSON):**

```
{
  "device_id": "esp32-cattle-001",
  "cattle_id": "C123",
  "timestamp": "2025-09-24T12:00:00Z",
  "temp": 38.6,
  "accX": 0.12,
  "accY": -0.04,
  "accZ": 9.78,
  "rumination_events": 14,
  "battery": 3.85,
  "status_flags": ["estrus_candidate"]
}
```

2. ESP32 Configuration & Provisioning

a) Initial Setup

- Device shipped with default firmware.

- Provisioned via **BLE** or **captive portal Wi-Fi**:
- Farmer opens Godhan app → selects collar.
- Sends Wi-Fi SSID, password, cattle_id.
- ESP32 saves config to NVS (non-volatile storage).

b) Config Parameters

- Wi-Fi SSID / Password
- MQTT broker URL + port
- Device ID, Cattle ID
- Sampling interval (default: 5 min)
- OTA update server URL

c) Update Methods

- **App-triggered OTA**: Admin pushes new firmware.
- **MQTT-triggered OTA**: Device subscribes to `fw/update/{device_id}`.
- **Manual reset**: Captive portal fallback if OTA fails.

d) Security

- Each device has **unique ID + secret key** provisioned.
- MQTT payload signed (HMAC-SHA256).
- TLS certificates pinned.
- Firmware updates signed + verified.

3. Backend APIs for ESP32 Communication

Device Registration

- **POST** `/api/devices/register`

```
{
  "device_id": "esp32-cattle-001",
  "cattle_id": "C123",
  "fw_version": "1.0.0",
  "battery": 3.9,
  "status": "active"
}
```

Data Upload (optional HTTP fallback)

- **POST** `/api/data`

```
{
  "device_id": "esp32-cattle-001",
  "cattle_id": "C123",
  "timestamp": "2025-09-24T12:00:00Z",
  "temp": 38.6,
  "accX": 0.12,
  "accY": -0.04,
  "accZ": 9.78,
  "rumination_events": 14,
  "battery": 3.85
}
```

Firmware Update Trigger

- GET /api/firmware/latest
- POST /api/devices/{id}/update

Alerts API

- POST /api/alerts

```
{
  "cattle_id": "C123",
  "type": "HEAT_DETECTED",
  "severity": "critical",
  "message": "Estrus detected. Insemination window open."
}
```

4. Example ESP32 Firmware Workflow

1. Wake from deep sleep.
2. Read sensors (IMU, temp, rumination).
3. Aggregate readings.
4. Connect to Wi-Fi.
5. Publish JSON payload to cattle/data/{device_id}.
6. Listen briefly for OTA command on cattle/cmd/{device_id}.
7. Enter deep sleep.

5. Key Topics Structure

- MQTT Topics:
- cattle/data/{device_id} → sensor data

- `cattle/cmd/{device_id}` → OTA / config commands
- `cattle/alerts/{cattle_id}` → backend → app notifications

- **QoS Strategy:**

- Data: QoS1 (important, but duplicate safe).
 - Alerts/Commands: QoS2 (exactly once).
-